Understanding Code Composition and Library Usage in Android Applications

Samuel Dubuisson
Inria / Univ. Lille
France
samuel.dubuisson@inria.fr

Romain Rouvoy
Univ. Lille / Inria
France
romain.rouvoy@inria.fr

Abstract

Android applications today combine complex, custom code with a growing number of third-party libraries, raising questions about their maintainability, efficiency, and security. In this study, we systematically measure the code composition of Android apps, focusing on the share of app-specific versus reused library code. We analyze a large dataset of applications categorized by size, popularity, and user ratings to identify patterns in code reuse and complexity. Additionally, we propose a probabilistic method to detect obfuscated code, improving the accuracy of static analysis for Android apps.

Our findings indicate that applications still contain a significant share of non-specific code, suggesting opportunities for further code deduplication and reuse strategies. These observations provide a clearer understanding of the Android app ecosystem and can inform future research on performance optimization, code security, and sustainable software practices.

ACM Reference Format:
Samuel Dubuisson and Romain Rouvoy. 2024. Understanding Code Composition and Library Usage in Android Applications. In *Proceedings of Conference Full Title (Conference Name '24)*. ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

Preface

This section serves as an introduction to the broader context, It is solely intended for this apprenticeship report.

Subject Context

For nearly 20 years, since the release of the iPhone in 2007, smartphones have become essential objects in the daily lives of many of us. These devices are highly complex and are among the most advanced technological items humans have ever created. One major area of complexity is miniaturization. This complexity requires the use of a vast array of different and rare materials, notably critical minerals [19], which pose social and environmental challenges. However, we do not keep our phones for life and tend to replace them regularly. On average, we change them every 3 years, according to several studies conducted in recent years.[1,2]

One of the main reasons why people change their smartphone is the lack of storage space. According to a 2016 Western Digital study,[3] more than half of smartphone users run out of memory every week in India—the World most populated country—forcing them to delete apps and content to free up space. The usual response to this problem has been to increase the memory capacity of devices. For example, from late 2017 to late 2019, the average NAND memory size grew from 43GB to 83GB .[4] However, we aim to propose an alternative solution. One of the main reasons for this lack of storage is the increasing size of applications. An ideal approach would be to reduce their size. Indeed, the average size of Android apps has steadily increased in recent years. For instance, the size of popular apps, such as *Slack* and *Zoom* has grown from about 10MB to around 60MB, while video game apps have expanded from an average of 45MB to several hundred MBs. *Facebook*'s app has also seen its size increase by more than 600% .[5] Several factors may explain this trend, such as the increasing complexity of apps and their code, but we believe that the main cause lies in the growing reliance on libraries. They offer reliable content, as they are generally considered secure and heavily tested, but they are sometimes used without much consideration and can embed a lot of unused functionalities. Through this work, we aim to measure how representative libraries are in the total code base, and how they are used.

Apprenticeship Context

My apprenticeship is taking place during my second year of the Master's program in Software Engineering at the University of Lille. I am working at Inria, a French national research institution dedicated to digital sciences and technology. Specifically, I am part of the Spirals project-team, which focuses on topics such as software evolution problems, energy consumption in software and AI, as well as cybersecurity.

Previous Work

Our initial objective was to measure and compute similarity scores between Android applications to evaluate how much code they have

[1] https://www.sellcell.com/blog/how-often-do-people-upgrade-their-phone-2023-statistics/
[2] https://mediabrief.com/strategy-analytics-report-on-smartphone-replacement-cycle
[3] https://www.pcquest.com/wd-survey-reveals-increase-in-smartphone-usage-leading-to-growing-storage-needs/
[4] https://www.counterpointresearch.com/insights/average-storage-capacity-smartphones-cross-80gb-end-2019/
[5] https://www.techradar.com/pro/Heres-why-your-smartphone-storage-is-disappearing-so-fast

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference Name '24, Location
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/24/XX
https://doi.org/XXXXXXX.XXXXXXX

in common. A high similarity score typically indicates that two apps share a significant amount of code. We drew inspiration from the work of Sacha Brisset on web page comparison [3], in which he proposed a new algorithm, named *Similarity-based Flexible Tree Matching* (SFTM), to compute similarity scores between web pages from both structural and semantic perspectives.

The aim of our project was to adapt this methodology to Android application packages (APKs). In our context, the structure of a web page was mapped to the file hierarchy of an APK, while the properties of HTML tags were translated into defined file properties such as size, file type, and semantic hashes. Our work on this topic is available online.[6]

However, we encountered several limits, the most significant of which were:

- Scalability issues: To compute similarity scores, each pair of APKs had to be compared, resulting in polynomial complexity. Even with clustering techniques—where new APKs are compared only to members of a specific cluster—the performance remained unsatisfactory, as we would reach a linear logarithmic complexity. Obtaining meaningful results would require a huge amount of computing power.
- Preserving semantics in the comparison: For the comparison to be meaningful, it was crucial to preserve semantic information across files. However, directly comparing all files is not feasible. A semantics-preserving hashing function would be ideal, but we were unable to identify a consistent and reliable method applicable for the different file types.
- Map the algorithm to the Android use case. Several aspects differ from the original Web page case. One of the most relevant is that the tree structure of the Android application hierarchy is not especially relevant to study, contrary to the DOM Tree of a web page that defines the page.

We decided to address the problem more directly by analyzing the internal composition of APKs. This involved measuring the proportion of original code versus third-party libraries, identifying how many libraries were used, and examining how those libraries were integrated and reused across applications.

Transition to the paper

. Our paper is made of three research questions. The experimental process is well defined and developed for RQ1 and RQ2, and we already obtained preliminary results. The experimental process is still under development for RQ3. We cannot present results for this research questions, but we present precisely our considered and already defined algorithms. Let us now dive into the concrete study.

1 Introduction

The broad subject has already been introduced in Section . For the paper, the referred section will be summarized here, with academic references. Let us also introduce our work topic more precisely here. We investigate techniques to detect and analyze code duplication in Android applications. For that, we decided to analyze the composition of Android applications to evaluate the proportion of code that is specific to the application and the proportion of code that is potentially repeated among other applications. To achieve this, we rely on static analysis techniques, and on an exploration of the file hierarchy of the Android application (more precisely, of the decompiled Android application). If we are able to obtain proportions of potentially repeated code, the distribution of libraires across applications and how they are used, we hope to be able to determine or estimate the amount of repeated information from a set or given number of applications.

1.1 Background

In this section, we address some key technical points that are worth explaining first. We discuss the foundations of the Android Operating System, the libraries, the Smali language, and obfuscation in Android applications.

1.1.1 Android. Android is the world's most widely used mobile operating system, powering billions of smartphones and tablets globally [1, 17]. It is built on a modified Linux kernel and supports applications written primarily in Java or Kotlin.

Apps are compiled into Dalvik bytecode (`.dex` files) and packaged into a ZIP-format archive with the extension `.apk`. This bundling process organizes the source code, resources, and assets into a single file for easier distribution and installation. The APK archive contains:

- `classes.dex`: the compiled code,
- resource files (XML, images),
- native libraries (in `.so` format),
- the `AndroidManifest.xml`, which declares components and permissions.

These files collectively describe the app's logic, resources, and structure.

For the purpose of static analysis and improved readability (especially since `.dex` files can be split into multiple files), an APK is often decompiled into Smali code.

1.1.2 Libraries. Android applications often rely on external or native libraries to extend functionality, reduce development time, or improve performance. These libraries can be written in Java/Kotlin and packaged in the file hierarchy; in our work, we name these libraries Android Libraries, they play a crucial role in our code analysis. Libraries could also be implemented in native languages like C/C++ and compiled into shared objects directory in the APK; we name them *Native Libraries*. Naturally, they offer better performance but are less handy to use as they need to be adapted to different phone architectures. Libraries serve various purposes, including networking (e.g., OkHttp), cryptography (e.g., Bouncy Castle), analytics (e.g., Firebase), or UI components. They can be open-source, proprietary, or bundled with development frameworks.

1.1.3 Smali. Our study mainly relies on Smali code, which will be formally introduced in the study in Section 2.4. To get an idea of what it looks like, here is a small snippet of Smali code. Interpreting its meaning is left to the reader (we trust them to successfully perform it).

```
.method public static main(Ljava/lang/String;)V
    .locals 2

```

[6] https://gitlab.univ-lille.fr/samuel.dubuisson.etu/pji

unpublished working draft. Not for distribution.

```
    const-string v0, "Hello, World!"

    invoke-static {v0}, Ljava/io/PrintStream;->println(
        Ljava/lang/String;)V

    return-void
.end method
```

1.1.4 Obfuscation. Android obfuscation regroups techniques used to make application code harder to analyze or reverse-engineer. Usually applied during the build phase, obfuscation changes the code's appearance while preserving its behavior. Its main purpose is to protect intellectual property, limit plagiarism or piggybacking, and in some cases to conceal malicious behavior. Obfuscation is common in both legitimate and malicious apps, with techniques like identifier renaming, string encryption, reflection, or code packing.

1.2 Literature Review

The composition of applications and its impact on their behavior has been widely studied in the literature. For example, Mojica *et al.* [14] examined application quality according to library usage, while Mojica *et al.* [15] further analyzed the impact of using specific types of libraries, such as advertising libraries.

Third-party libraries have received significant attention in research, with several studies focusing specifically on their identification and taxonomy. For instance, Li *et al.* [12], Backes *et al.* [2], Zhang *et al.* [26], and Wang *et al.* [20] have all contributed to cataloging and characterizing Android libraries.

Other works have focused on quantifying the presence of third-party libraries within applications. For example, Gonzalez *et al.* [6] measured the share of methods originating from libraries, while Wang *et al.* [21] investigated the proportion of third-party code. These studies often aim at identifying similarities between applications, with third-party libraries treated as an important confounding factor. Third-party libraries have also been examined in the broader context of code reuse, as in the work by Gonzalez *et al.* [7], and in studies centered on security concerns [2, 18].

Obfuscation remains a major challenge for Android static analysis, and has also been thoroughly considered in the literature. It must be accounted for when analyzing APKs [21], and has been studied both in terms of developing tools to detect obfuscation [10] and in understanding its role and prevalence in Android applications [4, 23, 27].

1.3 Research Gap

We target the study of application composition between application-specific and general content, which has not been covered by the state of the art, to the best of our knowledge, and to draw correlations with key application's properties. We also come with a new way of handling obfuscation for file classification, which is based on key findings from the literature.

Different applications come directly from this work. For example, if we detect a very high percentage of application-specific content in our application, we can think about methods to factorize the code in the smartphone by applying deduplication algorithms [24] on applications.

1.4 Research Questions

- **RQ1**: What proportion of an APK's content is app-specific versus general information?
- **RQ2**: Which application's properties impact its composition?
- **RQ3**: How third-party libraries are used inside the code?

RQ1 and RQ2 follow similar experimental setups, while RQ3 has its own. Then, our methodology sections are often composed of two parts.

1.5 Study Objectives

We aim to answer these research questions through a quantitative study, analyzing the application composition and behaviors.

We want to evaluate and measure the quantity of app-specific information, and code, in the application. We also aim to measure how and to what extent the libraries are used in the application, and if the library are used entirely or not in the application, as advanced by Backes *et al.* [2].

2 Methods

In this section, we define our global experience methodology. Our algorithms are detailed in the next section.

2.1 Research Design

RQ1. Our Research Design follows a classical quantitative study. We study the composition of Android applications according to a large pool of public applications.

RQ2. Our Research Design follows a an experimental (or causal-comparative) research design. We study specific groups and selected them according to their properties (size, popularity, notation) and a ranking (the most, the least and the medium) and want to see if the extreme groups of these properties have some specifications on their composition. We also compare the groups between them.

RQ3. Our Research Design follows a classical quantitative study. We aim to select a large pool of applications and study how they use their libraries more concretely.

2.2 Population & Sampling

We retrieved the applications from the classical stores, mainly from The Google Play Store. We decided to remove non-relevant applications. For that, we only kept the applications with at least 10, 000 downloads. We went from 4, 320, 406 applications to 394, 415 applications.

RQ1. We decided to select 600 randomly selected applications for a global analysis. This number ensures us significant results, while ensuring a fair and easy reproducibility.

RQ2. For each property and ordering selected, we selected 50 apps. It gives us a total of $9 \times 50 = 450$ apps.

RQ3. We aim to follow the same population and sampling as RQ1.

2.3 Variables Definition

RQ1 & RQ2. The related dependent variables are the different categories of files that constitute the application. These file categories are:

Unpublished working draft. Not for distribution.

- **Android Libraries** contain code from external Android libraries.
- **Native Libraries** are directly imported into the native code, supporting multiple architectures.
- **Business Code** is developed by the application team. Usually, we understand Business Code only as the code that follows the business rules. Here, it refers to the code **directly** developed by the team, which hence serves directly the business interest of the application.
- **Generated Code** during compilation. It includes, for example, annotations, serialization code, etc.
- **Resource Files** include fonts, images, sounds, and videos.
- **Android Configuration Files** are essential for OS compatibility.
- **Obfuscated Files** that were obfuscated with name overloading that cannot be normally classified.
- **Unknown Files** that cannot be categorized by the analyzer.

For a better understanding, we sometimes regroup these categories into groups that are relevant for our analysis. The categories are grouped as follows:

- **Specific Group:** Business Code, Resource Files, Generated Code, and Android Configuration Files. It represents the files that are strictly specific to the APK.
- **General Group:** Android Library Files and Native Library Files. It represents the files that could be potentially found in other APKs.
- **Unknown Group:** Unknown Files, Obfuscated Files.

In our study, we focus on Android code that is the most relevant to study developer behaviour. We draw the following groups, where we keep Business Code, Generated Code, Android Library Code, and Obfuscated Code.

2.3.1 RQ2. In RQ2, we analyze the correlations between App's composition and file properties. Hence, we follow the following independent variables: **Application's Size**, **Application's Popularity** (number of downloads), **User's Ratings**(stars on the store). Each of these variables is featured with an ordering: **High**, **Medium**, **Low**.

2.3.2 RQ3. For each application we studied, three dependent variables are considered: **the number of libraries**, **the number of methods per library**, and **the proportion of methods used in the application**.

2.4 Data Collection Tools

Retrieve the APKs. To retrieve the APKs, we used the AndroZoo dataset [**allix\textsc {AndroZoo}CollectingMillions2016**], self-defined as a "*collection of Android Applications collected from several sources, including the official Google Play app market*". It lists more than 25 millions APKs today. To retrieve specific apks according to criteria we defined, we also used the AndroZoo Metadata database [**alecci\textsc {AndroZoo}RetrospectiveGlimpse2024**].

Preparing APKs for Analysis. To analyze the contents of Android applications, we first needed to make their internal structure readable. For this, we used Apktool[7], a popular tool that disassembles APKs into more accessible formats, most notably Smali [5] code. Smali is an intermediate, human-readable representation of the Dalvik bytecode found in .dex files. It exposes the app's low-level logic, including class definitions, method instructions, and the relationships between them. This format is especially useful for static analysis, as it lets us understand what the app does without executing it. Once an APK is decompiled with Apktool, its structure typically looks like this:

```
<apk_name>/
|-- AndroidManifest.xml
|-- res/
|   `-- ... (XML layout files, images, etc.)
|-- \textsc{Smali}/
|   |-- com/
|   |   |-- example/
|   |   |   `-- myapp/
|   |   |       |-- MainActivity.\textsc{Smali}
|   |   |       `-- Utils.\textsc{Smali}
|   |   `-- google/
|   |       `-- firebase/
|   |           `-- analytics/
|   |               `-- FirebaseAnalytics.\textsc{Smali
|   |   |`-- ... (other packages)
|   |-- org/
|   |   |`-- apache/
|   |       `-- commons/
|   |           `-- IOUtils.\textsc{Smali}
|   |-- kotlin/
|   |   `-- collections/
|   |       `-- ArraysKt.\textsc{Smali}
|   `-- ... (other packages)
|-- \textsc{Smali}_classes2/
|   |-- ...
|-- lib/
|   `-- armeabi-v7a/
|       `-- libnative-lib.so
|-- assets/
|-- unknown/
`-- original/
```

2.5 Statistical Methods

The studied population is very large: to verify the significance of our results on the general application composition, we used the Cochran Formula. For 95% of significance and 5% accepted margin, we need to study at least 384 applications.

For the group comparisons, we selected 50 applications in each group. We verified if the comparisons with the others were significant. For that, we first verify if the obtained results followed a normal law. If the obtained results followed a normal distribution, we used the t-test for comparing two groups or ANOVA (Analysis of Variance) for comparing multiple groups. If the normality assumption was not met, we applied non-parametric tests such

[7] https://github.com/iBotPeaches/Apktool

Unpublished working draft. Not for distribution.

as the Mann-Whitney U test for two-group comparisons or the Kruskal-Wallis test for multiple groups.

2.6 Ethics & Approval

This study did not involve human participants or personal data. The analysis was conducted on publicly available application data with no sensitive personal information.

3 Algorithms & Processes

In this section, we detail the algorithms developed and used to answer our research questions.

3.1 RQ1 & RQ2

More specifically, we first:

(1) Retrieved metadata and stored it in a CSV file. The apps are retrieved according to given criteria: a property, a sorting, a number and potentially a developer. For RQ1, they are just retrieved randomly.
(2) For each app, download, dissassemble it, and run the categorization algorithm on it.
(3) Computed and stored file category proportions.
(4) Depending on the desired visualization, deobfuscate the results.
(5) Visualized results as plots.

Categorization algorithm. We categorized files based on their directory paths. Here is an overview of our general categorization rules, before diving into the ambiguous cases:

- **Business Code:** The file is contained in the `smali/` folder and has the package name in its file path.
- **Generated Code:** Some packages within the application are not considered business code strictly written by the developer team but contain generated code. In some cases, these codes are fully generated, while in others, they are only partially generated. To handle such scenarios, we implemented specific rules to precisely classify these files. For that, we drew a whitelist of the well-known elements of Android Applications that contain generated code. For the parts that are only partially generated, we drew a whitelist of files that are generated. In it, we have—for example—Dagger, Hilt and Protobuf.
- **Android Libraries:** The file is in the `smali/` folder but does not contain the package name in its file path.
- **Native Libraries:** The file is in the `lib/` folder.
- **Resource Files:** The file is in the assets or `res/` folder.
- **Android Configuration Files:** `Android Manifest.xml` file or if the file is the Android package in the `smali/` folder, or the original folder.
- **Obfuscated Files:** Files with meaningless names with three or less three characters following the approach used by Ikram *et al.* [10]. We excluded keywords, such as "`com`", "`sdk`" or "`app`".
- **Unknown Files:** The file is in the `Unknown/` folder.

How to classify obfuscated files?

Different possibilities. For more precise results considering the code distribution, we wanted either to deobfuscate it precisely or to obtain a good heuristic to classify them. We found impossible to deobfuscate directly the code and classify each file precisely. Researchers present their tool for deobfuscation, such as [23] with Obfuscan and [27] with ApkDeGuard. ApkDeGuard reports for example a 80% precision, that is good but does not ensure very precise results. Moreover, it only offers a web interface to use it, that is very hard to include in an efficient process. We regret that the most efficient deobfuscator or obfuscator detector tools, that looks powerful, are not open source. They might stayed closed sources to prevent malware creators to anticipate deobfuscation techniques.

Key findings in the literature. We use key results found by Wermke *et al.* [23]. They found that only 25% of the developers intentionally obfuscate their apps. These results are confirmed by Zhang *et al.* [27] who write that the majority of the applications are not intentionally obfuscated, but contain obfuscated libraries. Wermke also found that 64.7% of the applications were obfuscated, but only 24.9% were obfuscated in the main package: the package containing the business code. However, Wermke reports that one key property influencing these results is the app popularity. The most downloaded apps are twice as obfuscated by the developer. It can be explained by the fact that they understand better the benefits from obfuscation and know better how to do it as they develop applications more often. We should take that into account in our results.

Our contribution. We solve the obfuscation issue following a probabilistic solution.

Let us firstly consider the naive Hypothesis H_1: *"The different categories are equivalently distributed in the app and each of them obfuscate equally"*.

Let us consider OM the following event: a given obfuscated file is not in the main package. The associated probability is

$$P(OM) = \frac{0.647 - \frac{0.249}{2}}{0.647} \times 100 = 0.8.$$

The opposite event is written $ON = \overline{OM}$ and represents: a given obfuscated file is in the main package. The associated probability is:

$$P(ON) = 1 - P(OM) = 0.2$$

From that, we obtain the following distribution for obfuscated files:

- Business code: P(OM) = 0.2
- Generated code: P(ON) / 2 = 0.4
- Library code: P(ON) / 2 = 0.4

However, to be more realistic, we need to get rid of the first part of the hypothesis: **all categories are not equally distributed**. To overcome that, the first step is to distribute proportionally between Generated code and Library code.

Let us say that, among all the Android code, b is the proportion of business code, g the proportion of generated code and l the proportion of library code:

- Business code: $P(OM)$
- Generated code: $P(ON) \times \frac{g}{g+l}$
- Library code: $P(ON) \times \frac{l}{g+l}$

Unpublished working draft. Not for distribution.

Now, we also have to take into account the distribution probability with the global distribution of assets. For that, we multiply our initial proportions by the ratio of presence in the code for each category. Then, we normalize our results to obtain percentages (as OM and ON form a partition of the sample space).

The share of obfuscated code that is considered as business code is computed as follows:

$$\frac{b \times P(OM)}{b \times P(OM) + (g+l) \times P(ON)}$$

For the generated code, the share is:

$$\frac{(g+l) \times P(ON)}{b \times P(OM) + (g+l) \times P(ON)} \times \frac{g}{g+l}$$
$$= \frac{g \times P(ON)}{b \times P(OM) + (g+l) \times P(ON)}$$

For the library code, the share is:

$$\frac{(g+l) \times P(ON)}{b \times P(OM) + (g+l) \times P(ON)} \times \frac{l}{g+l}$$
$$= \frac{l \times P(ON)}{b \times P(OM) + (g+l) \times P(ON)}$$

Finally, our hypothesis simply becomes H_2: **"Each category of the application is evenly obfuscated"**. We found this hypothesis reasonable as we did not find any way to refute it.

To compute the actual proportions with highly downloaded (>10M) and lowly downloaded apps, we just have to adjust the value of $P(OM)$ to respectively 0.4 or 0.1 and $P(ON)$ to 0.6 or 0.9, according to the results from Wermke *et al.* and Zhang *et al.* When we study the links between application composition and application popularity, we should apply these different calculations for the most popular and least popular applications.

From this probabilistic approach, we can infer the real type of obfuscated types and obtain better results.

3.2 RQ3

Our approach follows a static analysis, where we examine the code of the application without actually running it. Zeller *et al.* [25] presented static analysis as deductive reasoning, where concrete properties of the program are drawn from an abstract analysis. We chose static analysis for three main reasons:

- The study case is well suited for static analysis: We aim to list the libraries and the libraries' methods, the calls to these libraries, and method dependency graphs. Static analysis answers well to these cases.
- On performance and reproducibility issues: Static analysis is far more convenient. Dynamic analysis requires running the applications and inducing behaviors.
- Static analysis generally holds convincing results: As Onwuzurike *et al.* [16] showed in a related field (malware detection) in Android, it is at least as efficient as dynamic analysis.

More precisely, we first proceed to list libraries and methods from each application. It can easily be done by working on the file hierarchy, as we did in RQ1 and RQ2, or by studying the Android Manifest.

For one APK, our algorithm follows the following process, and takes as a parameter a specific library:

(1) Identify all the methods in the APK.
(2) Isolate the methods from the application-specific code, the specific library, and other libraries (using our work from RQ1).
(3) Construction of a dependency tree for the specific library methods.
(4) List all the calls from the app-specific code towards the specific library.
(5) List all the methods that are called from the code using the dependency tree.

We identify the method calls and definitions by analyzing the Smali code.

A Smali method is defined as follows:[8]

```
.method <access_flags> <name>(<parameter_types>)<
    return_type>
   <code_block>
.end method
```

Methods invocations are following this pattern:

```
invoke-<type_of_invocation> {<parameters>}, <class_name
    >;-><method_name>()<type>
```

Different invocation types exist (virtual for public methods, static for static methods, etc, direct for constructors, etc.), but do not influence our analysis. We are more interested in the target class name that allows us to guess the origin of the method and the method name.

We aim to extend the algorithm to take as input a set of applications and a set of library names to support an inter-application study. It can help us to find how many libraries are used on average in the applications, and the share of the libraries that are effectively used. If this proportion tends to 100%, it means that libraries are always partially included in the application to satisfy only what is needed, and so that tree checking algorithms are applied.

3.3 Validity & Reliability

The algorithms are developed with Python. It ensures a clear development and an easy abstraction of the implemented rules. We developed a suite of tests using Pytest to ensure the correctness of our algorithms.

RQ1 & RQ2. For our categorization algorithm, a similar method, based on file hierarchy, has also been used by Li *et al.* [11] in their third-party library study. We ensured that the categorization algorithm specified in Section 3.1 runs a fair number of tests on specific and special cases.

RQ3. Our algorithm is also validated by tests on specific and special cases. Analyzing the Smalli code to infer application behaviour is a common way of operating. For example, Hoffman *et al.* [8] also chose this technique to automatically detect malware in Android applications.

4 Results

We obtained preliminary results for RQ1 and RQ2. As the experimental process is not finalized for RQ3, we did not obtained results yet for this research question.

[8] https://pysmali.readthedocs.io/en/latest/api/smali/language.html#methods

Unpublished working draft. Not for distribution.

4.1 RQ1

In this section, we present the results we obtain for RQ1: we study the application and code composition. We chose to plot our results using bar charts, as they are well adapted for showing proportions and offer clearer results than pie charts.

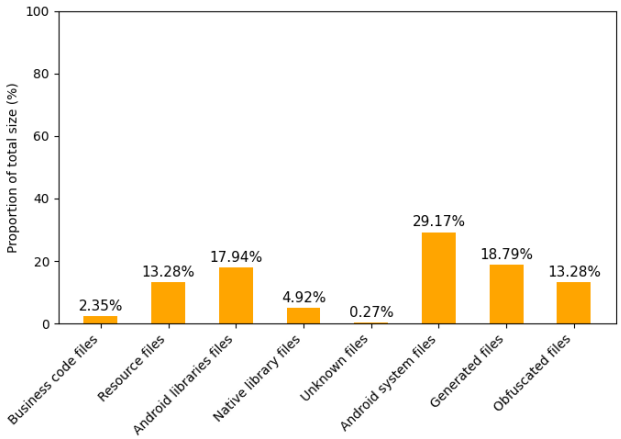

Figure 1: Distribution on Application Composition with Obfuscation

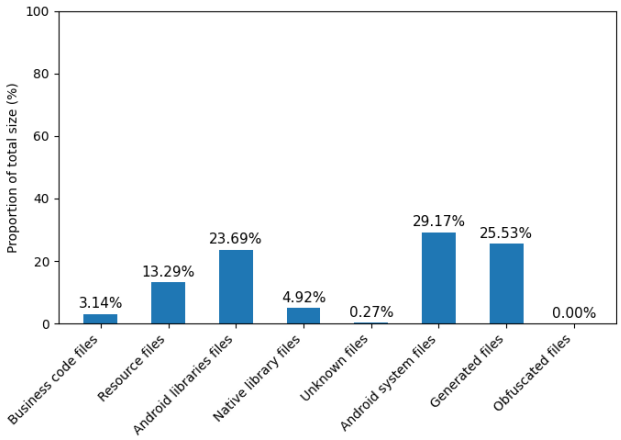

Figure 2: Distribution on Application Composition

The application composition. Figures 1 and 2 depict the distribution of categories across 600 randomly selected applications in dataset. From Figure 1 to Figure 2, we see the effects of deobfuscation: among the 13.28% of obfuscated files, 6.6 points has been transferred to generated files, 5.4 points have been transferred to Android Libraries, and 1.3 points to Business Code Files.

Following our classification of categories, we have:

- **App-Specific Categories**: Business code files + Resource files + Android System Files + Generated code = 3.60% + 13.29% + 29.17% + 25.41% = 71.47%
- **General Categories**: Android libraries files + Native Library Files = 23.34% + 4.92% = 28.26%
- The rest is the part of unknown files: 0.27%

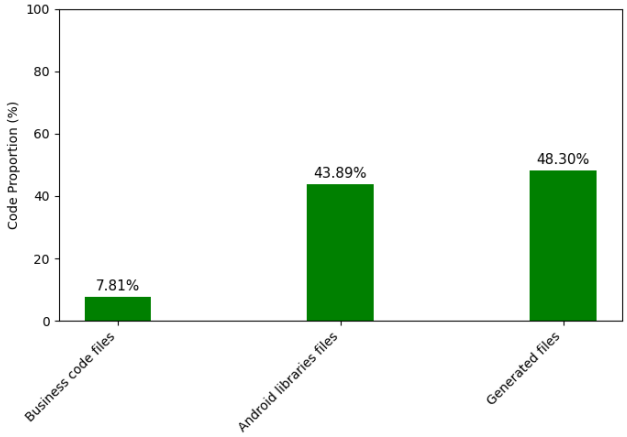

Figure 3: Proportions on Code Composition

4.1.1 The code composition. In this subsection, we look more precisely at the code composition. First, we observe that the business code (the one we defined as being strictly written by the developers' team) is not highly represented (less than 8%). When adding Generated Code, we reach a total of 56% of application-specific code against 44% of Android libraries.

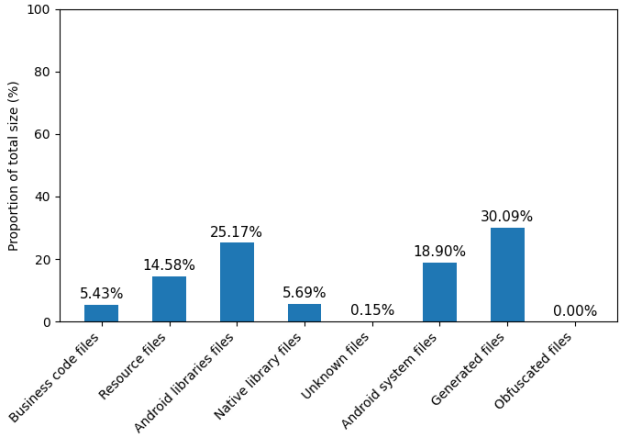

Figure 4: Weighted proportions on application composition

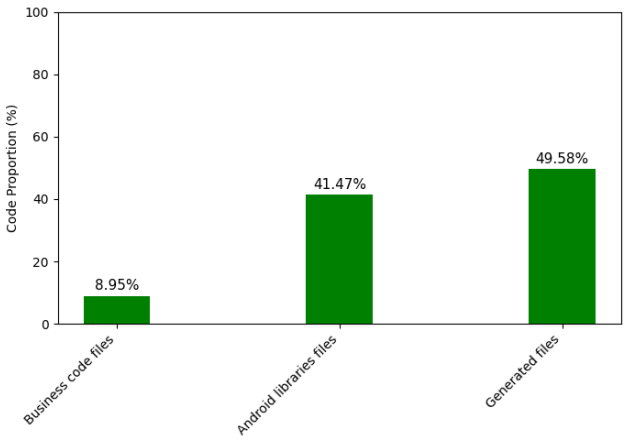

Figure 5: Weighted proportions on code composition

4.1.2 A different average. This section debates another way of computing the average proportions for each category serves as a transition to the next research question. In our first results, to compute the average proportions, we considered all applications evenly, giving us insightful results on the developer behaviours and what the average applications look like. Here, we weighted our results. Hence, the biggest applications have the most impact on average. Why would we interest to this way of calculating? It gives us more insightful results if we are interested in the potentiality of observing duplicated information across the applications, as here, we directly consider the part of memory in the phone that is dedicated to the considered categories.

First, we observe that Android system files are far more represented in this new average way of computing: it drops from 29% in Figure 2 to 18% in Figure 4. On the other hand, code content increases. All code categories are more represented, and the business code proportion among code is also higher (from Figure 5, we observe that the share of business code file is equal to 8.9%, against 7.8% in Figure 3). We can infer that larger application contains more code proportionally than others, and even more business code, and that the Android configuration is more or less constant in each application. With this new way of computing, we get 69% of app-specific content.

4.2 RQ2

In this section, we will be interested in the relations between applications properties and their composition. We already obtains some responses with the last subsection regarding size. For now, we have results on specific content vs. general content in code only. We aim to enlarge the results to all application content also. We will need to come up with more elaborated visual representations for that. We chose a clear and simple table for code content, with properties as columns and ordering as rows.

	Best	Medium	Worst
Download	0.52	0.54	0.53
Grade	0.56	0.58	0.50
Size	0.65	0.59	0.64

Table 1: Share of specific code content across app categories

4.2.1 Popularity. We do not remark high variations of app-specific code between the three different groups. The medium group has slightly more app-specific code, but 1 or 2 points does not look significant enough.

4.2.2 User Ratings. The results are different when it comes to user ratings. We observe that the least appreciated got significantly less app-specific code: 50%, 6 and 8 less points that the others groups.

4.2.3 Size. An interesting phenomenon appears for the size property. While we thought in RQ1 results that the largest applications had the most app-specific code, we observed here that both largest and lightest applications have the most app-specific code. Actually, it fits well our first results, as the lightest applications had nearly no impact with our way of calculating in Section 4.1.2.

4.3 Statistical Significance

We selected a random sample of 600 applications out of several millions. According to Cochran's formula, a minimum of 384 samples is required to achieve a 5% margin of error with 95% confidence. Our sample of 600 reduces the margin of error to approximately 4%, ensuring that our results are statistically significant and representative of the general application pool.

We have not run the multi group comparisons yet to verify significance. Moreover, we plan to increase the application pool.

5 Discussions

In this section, we offer an interpretation of the results we obtained. Then, we compare them with the literature results we found. Finally, we explain some limitations of your studies and the future research directions.

5.1 Results Interpretations

In our applications, we observe that a large portion of the content is specific (around 70%, depending on the calculation method). This still implies that approximately 30% could be shared with other applications, meaning that a significant amount of memory is potentially *wasted* by storing duplicated data. Considering the implementation of deduplication algorithms in our Operating Systems [22] could therefore be important.

We also observed relationships between application properties and their code composition, particularly regarding size and grade. Both very lightweight and very large applications contain a high proportion of app-specific code (more than two-thirds). This suggests that the largest applications are not simply large bundles of libraries, while the smallest ones cannot rely extensively on external libraries. Conversely, we found that the least appreciated applications have the lowest proportion of app-specific code. We

Unpublished working draft. Not for distribution.

hypothesize that these applications often consist of large amounts of library code without producing anything of real interest for the end user. Highly appreciated applications still make use of Android libraries, as these provide valuable security features and reliable functionality.

5.2 Comparison with Literature

Li *et al.* [11] evaluate the average amount of library code to 60% in Android applications, with only 42% of applications with more library code than application-specific code. Hence, their average is way higher than their median, letting us think that this very high average is due to extreme cases and applications with almost only library code.

5.3 Study Limitations and Future Research Directions

This section consists in key study limitations that we found and pave the way for future works.

RQ1 & RQ2. We can improve the Generated code detection by refine and expand the list of well-known generated package. Also, for even more related results, we could run the experiments with a bigger population, especially for RQ2, that run the experiments on a lot of different subgroups, but it asks more time and computing power. Specifically on RQ2, our causal comparative study partially limits us in our interpretations: we could consider running a correlational study, that can help us to draw strict correlations between property and composition. We could also consider other property to study, as the developer team/company that developed the application—studying a predefined pool of developers—or the application category.

RQ3. One key limitation when only using static analysis is that we are missing the dynamic calls. They are often found when the app is obfuscated [4]. We might consider partially using dynamic analysis to catch them. We could also be interested in comparing the usage of a same library across several applications. We might also wonder to what extent libraries of same versions strictly contain the same code across applications, and verify to what extent they could be slightly modified by the developer team. The idea is defended by Backes *et al.* [2], while other studies suppose the opposite, meaning that libraries are always found in the same form, as the one by Whang *et al.* [20], or found than the vast majority (more than 80%) of Android Libraries were used with no modification [9].

6 Conclusion

In this work, we presented an in-depth analysis of the internal composition of Android applications, focusing on the proportion of application-specific code compared to shared libraries. Our study highlighted that a significant share of code is still specific to each application, even though many apps integrate third-party libraries. This suggests opportunities for more advanced reuse or deduplication approaches, potentially reducing app sizes and improving performance.

We also examined the relationships between applications characteristics such as download count, user ratings and size—and their code composition. The results show that larger and better-rated apps tend to contain more application-specific code, whereas less popular apps rely more heavily on external libraries. This may reflect the higher degree of customization and functionality expected by users of successful apps.

Additionally, we proposed a probabilistic method to identify and classify obfuscated code within applications, which is a major challenge for static analysis. This technique still needs to be validated on larger datasets, and could be extended with dynamic analysis in the future to better understand real execution behaviors.

In future work, we plan to expand this study by completing experiments on RQ3, to better analyze the structure and relationships of shared libraries across applications. Studying developer practices around library reuse and updates would also help to understand their impact on code quality, security, and maintainability.

Apprenticeship Reflections and Perspectives

Like Section , this last section expands on the apprenticeship report.

Acknowledgements

Firstly, I would like to sincerely thank all the members of the Spirals project-team for their warm welcome and for fostering such a supportive and stimulating work environment. Being part of such a dynamic and collaborative group has been both enriching and motivating. I am especially grateful to Romain Rouvoy, who first introduced me to the team last year through a university project, and later offered to supervise me during my apprenticeship. He provided valuable supervision and insightful advice throughout both experiences, advising me on research directions and helping me develop the best experimental processes. I would also like to warmly thank Clément Quinton, who supervised me during my Master's research thesis; the advice he gave me at that time proved very useful again in this project, especially for the writing and the literature review. I am also thankful to Charles Paperman for his university supervision; the various meetings we had together challenged our progress efficiently and helped us to move further in our work. I would also like to thank Sophie Cerf and Adrien Luxey for their valuable supervision on another topic I had the opportunity to work on, that I detail in the next section.

Other Works during the Apprenticeship

After expanding on the work from the university project during the first few months of the apprenticeship (which eventually led us to the subject presented in this paper) I also became involved, from early 2025, in a project exploring the materiality of digital technologies through a systemic approach.

In this project, we aim to study the concrete impact of digital technologies on the economy and the environment, examining aspects such as the use of minerals, energy, water, and consumer and producer behaviours, from a systemic perspective (it allows for example to measure efficiently rebound effects). I had the opportunity to study Meadows' book [13] during this time, which provided a valuable theoretical foundation.

As part of this work, we met with economists and geologists in Grenoble to further develop and refine our ideas. Engaging with researchers working on related topics has been a great source of motivation and inspiration.

Unpublished working draft. Not for distribution.

Other noticables Experiences

I also attended the Green Days Conference in Rennes in March 2025, a French event dedicated to Green IT concerns. It was the first conference I participated in, giving me the opportunity to discover related topics and meet researchers working in this area.

In June 2025, I discovered my first international conference: the DisCoTec conference, which focuses on distributed systems.

In addition, I successfully completed the SST (*Sauveteur Secouriste du Travail* / Workplace First-aider) training and obtained the certification. I am grateful to Inria for providing access to this course, as having trained first-aiders is essential yet still not widespread enough in many companies.

Personnal Reflection

I have been interested in research since my early years at university. That curiosity led me to choose a research-oriented track during my bachelor's, which I really enjoyed. Later on, I did two internships in IT service companies—both were great learning experiences and helped me build solid technical skills. However, I felt the need to experience academic research more directly. When I finally had the opportunity to immerse myself in a real research environment, it truly confirmed my passion for the field. I enjoyed the freedom to explore ideas, the challenge of solving difficult problems, and the inspiring conversations with researchers.

Perspectives

I am very pleased to share that I will be continuing with the team next year as a PhD student, working on the materiality of digital technologies though a systemic approach, one of the topics I have explored this year. Our thesis funding has just been confirmed as I write these lines. I look forward to deepening my work on this subject, and I am sure this new research experience will be at least as enriching as the first one. I will do my best to apply what I have learnt during this year of apprenticeship to start my thesis on a strong footing.

References

[1] *Android OS Market Share Worldwide.*

[2] Michael Backes, Sven Bugiel, and Erik Derr. "Reliable Third-Party Library Detection in Android and Its Security Applications". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. Vienna Austria: ACM, Oct. 2016, pp. 356–367. ISBN: 978-1-4503-4139-4. DOI: 10.1145/2976749.2978333. (Visited on 06/15/2025).

[3] Sacha Brisset et al. "SFTM: Fast Comparison of Web Documents Using Similarity-based Flexible Tree Matching". In: (2020). DOI: 10.48550/ARXIV.2004.12821. (Visited on 12/03/2024).

[4] Shuaike Dong et al. *Understanding Android Obfuscation Techniques: A Large-Scale Investigation in the Wild*. Jan. 2018. DOI: 10.48550/arXiv.1801.01633. arXiv: 1801.01633 [cs]. (Visited on 06/17/2025).

[5] Jesus Freke. *Smali: An Assembler/Disassembler for Android's Dex Format.*

[6] Hugo Gonzalez, Natalia Stakhanova, and Ali A. Ghorbani. "DroidKin: Lightweight Detection of Android Apps Similarity". In: *International Conference on Security and Privacy in Communication Networks*. Ed. by Jing Tian, Jiwu Jing, and Mudhakar Srivatsa. Vol. 152. Cham: Springer International Publishing, 2015, pp. 436–453. ISBN: 978-3-319-23828-9 978-3-319-23829-6. DOI: 10.1007/978-3-319-23829-6_30. (Visited on 12/03/2024).

[7] Hugo Gonzalez, Natalia Stakhanova, and Ali A. Ghorbani. "Measuring Code Reuse in Android Apps". In: *2016 14th Annual Conference on Privacy, Security and Trust (PST)*. Auckland, New Zealand: IEEE, Dec. 2016, pp. 187–195. ISBN: 978-1-5090-4379-8. DOI: 10.1109/PST.2016.7906925. (Visited on 02/03/2025).

[8] Johannes Hoffmann et al. "Slicing Droids: Program Slicing for Smali Code". In: *Proceedings of the 28th Annual ACM Symposium on Applied Computing*. Coimbra Portugal: ACM, Mar. 2013, pp. 1844–1851. ISBN: 978-1-4503-1656-9. DOI: 10.1145/2480362.2480706. (Visited on 07/06/2025).

[9] Wenhui Hu et al. "Duet: Library Integrity Verification for Android Applications". In: *Proceedings of the 2014 ACM Conference on Security and Privacy in Wireless & Mobile Networks*. Oxford United Kingdom: ACM, July 2014, pp. 141–152. ISBN: 978-1-4503-2972-9. DOI: 10.1145/2627393.2627404. (Visited on 07/05/2025).

[10] Muhammad Ikram, Pierrick Beaume, and Mohamed Ali Kaafar. *DaDiDroid: An Obfuscation Resilient Tool for Detecting Android Malware via Weighted Directed Call Graph Modelling*. Aug. 2019. DOI: 10.48550/arXiv.1905.09136. arXiv: 1905.09136 [cs]. (Visited on 06/22/2025).

[11] Li Li et al. "An Investigation into the Use of Common Libraries in Android Apps". In: *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. Vol. 1. Mar. 2016, pp. 403–414. DOI: 10.1109/SANER.2016.52. (Visited on 02/12/2025).

[12] Menghao Li et al. "Large-Scale Third-Party Library Detection in Android Markets". In: *IEEE Transactions on Software Engineering* 46.9 (Sept. 2020), pp. 981–1003. ISSN: 1939-3520. DOI: 10.1109/TSE.2018.2872958. (Visited on 06/17/2025).

[13] Donella H. Meadows and Diana Wright. *Thinking in Systems: A Primer*. Nachdr. White River Junction, Vt: Chelsea Green Pub, 2011. ISBN: 978-1-60358-055-7.

[14] Israel J. Mojica et al. "A Large-Scale Empirical Study on Software Reuse in Mobile Apps". In: *IEEE Software* 31.2 (Mar. 2014), pp. 78–86. ISSN: 1937-4194. DOI: 10.1109/MS.2013.142. (Visited on 02/14/2025).

[15] Israel J. Mojica Ruiz et al. "Impact of Ad Libraries on Ratings of Android Mobile Apps". In: *IEEE Software* 31.6 (Nov. 2014), pp. 86–92. ISSN: 1937-4194. DOI: 10.1109/MS.2014.79. (Visited on 02/14/2025).

[16] Lucky Onwuzurike et al. "A Family of Droids-Android Malware Detection via Behavioral Modeling: Static vs Dynamic Analysis". In: *2018 16th Annual Conference on Privacy, Security and Trust (PST)*. Aug. 2018, pp. 1–10. DOI: 10.1109/PST.2018.8514191. (Visited on 06/11/2025).

[17] Meier Reto. *Professional Android 4 Application Development*. Wrox. 2012. ISBN: 9780470560237.

[18] Mengtao Sun and Gang Tan. "NativeGuard: Protecting Android Applications from Third-Party Native Libraries". In:

Unpublished working draft. Not for distribution.

Proceedings of the 2014 ACM Conference on Security and Privacy in Wireless & Mobile Networks. Oxford United Kingdom: ACM, July 2014, pp. 165–176. ISBN: 978-1-4503-2972-9. DOI: 10.1145/2627393.2627396. (Visited on 07/02/2025).

[19] *Untangling the Critical Minerals Knot: When ICT Hits the Energy Transitions*. Dec. 2023.

[20] Haoyu Wang and Yao Guo. "Understanding Third-Party Libraries in Mobile App Analysis". In: *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. Buenos Aires, Argentina: IEEE, May 2017, pp. 515–516. ISBN: 978-1-5386-1589-8. DOI: 10.1109/ICSE-C.2017.161. (Visited on 02/14/2025).

[21] Haoyu Wang et al. "WuKong: A Scalable and Accurate Two-Phase Approach to Android App Clone Detection". In: *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. Baltimore MD USA: ACM, July 2015, pp. 71–82. ISBN: 978-1-4503-3620-8. DOI: 10.1145/2771783.2771795. (Visited on 02/14/2025).

[22] Guangyu Wei et al. "Sparrow: Flexible Memory Deduplication in Android Systems with Similar-Page Awareness". In: *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. Mar. 2024, pp. 1–6. DOI: 10.23919/DATE58400.2024.10546588. (Visited on 12/03/2024).

[23] Dominik Wermke et al. *A Large Scale Investigation of Obfuscation Use in Google Play*. Feb. 2018. DOI: 10.48550/arXiv.1801.02742. arXiv: 1801.02742 [cs]. (Visited on 06/19/2025).

[24] Qirui Yang, Runyu Jin, and Ming Zhao. "SmartDedup: Optimizing Deduplication for Resource-constrained Devices". In: ().

[25] Andreas Zeller. "Program Analysis: A Hierarchy". In: ().

[26] Xian Zhan et al. "Research on Third-Party Libraries in Android Apps: A Taxonomy and Systematic Literature Review". In: *IEEE Transactions on Software Engineering* 48.10 (Oct. 2022), pp. 4181–4213. ISSN: 1939-3520. DOI: 10.1109/TSE.2021.3114381. (Visited on 06/05/2025).

[27] Xiaolu Zhang et al. "Android Application Forensics: A Survey of Obfuscation, Obfuscation Detection and Deobfuscation Techniques and Their Impact on Investigations". In: *Forensic Science International: Digital Investigation* 39 (Dec. 2021), p. 301285. ISSN: 26662817. DOI: 10.1016/j.fsidi.2021.301285. (Visited on 06/17/2025).

Unpublished working draft.
Not for distribution.